

コンピュータビジョン

担当: 井尻 敬

Contents

01. 序論 : イントロダクション
02. 特徴検出1 : テンプレートマッチング、コーナー検出、エッジ検出
03. 特徴検出2 : ハフ変換、DoG, SIFT特徴
04. 領域分割 : 領域分割とは、閾値法、領域拡張法、グラフカット法、
05. オプティカルフロー : 領域分割残り、Lucas-Kanade法
06. パターン認識基礎1 : パターン認識概論、サポートベクタマシン
07. パターン認識基礎2 : ニューラルネットワーク、深層学習
08. パターン認識基礎3 : 主成分分析, オートエンコーダ
09. 筆記試験
10. プログラミング演習 1: PC室
11. プログラミング演習 2: PC室
12. プログラミング演習 3: PC室
13. プログラミング演習 4: PC室
14. プログラミング演習 5: PC室

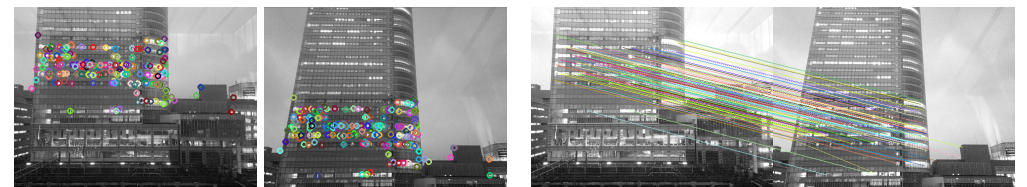
特徴点検出

- 特徴点とその応用例
- 復習: 線形フィルタ
 - Sobel filter
 - ガウシアンフィルタとその性質
- 特徴点とは
- SIFT特徴
- Hough変換

特徴点とは

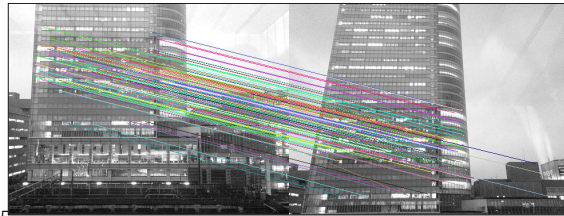
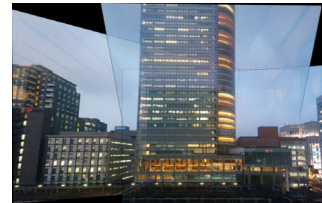
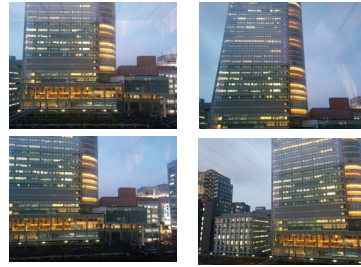
画像内から特徴的な点を検出し検出した点の局所的な特徴を比較できる形で記述したもの

画像内から特徴的な場所を検出し似た特徴を持つ場所と対応付けられると、、、
→パノラマ合成, ステレオ視, 物体認識, VR (位置あわせ), etc



応用例1 – パノラマ合成

- 一部が重なった写真を複数撮影
 - 特徴点を抽出し対応付け
 - 対応を満たすように変形して重ねる
- 1枚の大きな画像を作成できる

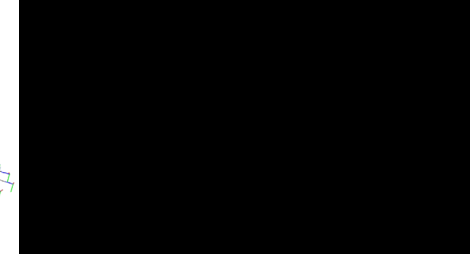


5

応用例2 – フォトグラメトリ

Photogrammetry: 複数写真から3Dモデルを生成する手法

- 物体や空間を複数の視点から撮影 (各写真が部分的には重なる必要あり)
- 各写真から特徴点を抽出し (SIFTもよく利用される)、特徴点のマッチングを計算
- 特徴点を3D空間に投影した際の辻褄が合うよう各写真の撮影視点や焦点距離を最適化
- 密な点群を発生させMeshモデルを構築する



特徴点検出

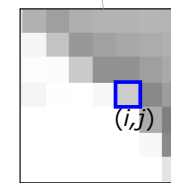
- 特徴点とその応用例
- 復習: 線形フィルタ
 - Sobel filter
 - ガウシアンフィルタとその性質
- 特徴点とは
- SIFT特徴
- Hough変換

7

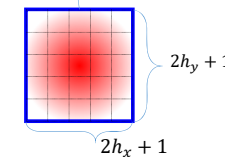
線形フィルタとは

出力画素値を周囲画素の重み付和で計算するフィルタ

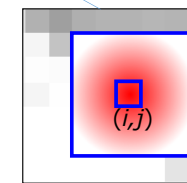
$$I'(i,j) = \sum_{m=-h_y}^{h_y} \sum_{n=-h_x}^{h_x} h(m,n) I(i+m, j+n)$$



$I'(i,j)$
出力画像



$h(i,j)$
フィルタ



$I(i,j)$
入力画像

復習

線形フィルタの例

復習



1/16	2/16	1/16
2/16	4/16	2/16
1/16	2/16	1/16



ぼかす

0	-1	0
-1	5	-1
0	-1	0



鮮鋭化

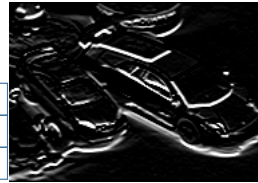


-1	0	1
-2	0	2
-1	0	1



横方向の微分を計算 $I_x = \frac{\partial I}{\partial x}$
sobel filter

-1	-2	-1
0	0	0
1	2	1



縦方向の微分を計算 $I_y = \frac{\partial I}{\partial y}$
sobel filter

線形フィルタ (convolution)

復習

連続 :

$$(g * f)(x) = \int_{-\infty}^{\infty} g(t) f(x - t) dt$$

離散 :

$$(g * f)(x) = \sum_{k=-\infty}^{\infty} g(k) f(x - k)$$

『*』を畳み込み積分 (Convolution) と呼び、以下の性質が成り立つ

$$\text{交換 : } g * f = f * g$$

$$\text{結合 : } f * (g * h) = (f * g) * h$$

$$\text{分配 : } f * (g + h) = f * g + f * h$$

$$\text{微分 : } \frac{d}{dx} (f * g) = \frac{df}{dx} * g = f * \frac{dg}{dx}$$

$$\text{フーリエ変換 : } \mathcal{F}(f * g) = \mathcal{F}(f)\mathcal{F}(g)$$

離散系では、カーネルが十分大きく境界付近でカーネルの値が0のフィルタ処理の際、この性質がおおよそ成り立ちます。

畳み込み積分のフーリエ変換: $\mathcal{F}(f * g) = \mathcal{F}(f)\mathcal{F}(g)$

復習

関数 f, g の畳み込み積分は以下の通り定義される,

$$h(x) = \int_{-\infty}^{\infty} f(x - t)g(t)dt$$

この h のフーリエ変換は以下の通り

$$\mathcal{F}[h(x)](\omega) = \int_{-\infty}^{\infty} \left(\int_{-\infty}^{\infty} f(x - t)g(t)dt \right) e^{-i\omega x} dx$$

$$= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x - t)g(t) e^{-i\omega x} dt dx$$

$$= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x - t)g(t) e^{-i(x-t)\omega} e^{-it\omega} dt dx$$

$$= \int_{-\infty}^{\infty} \left(\int_{-\infty}^{\infty} f(x - t) e^{-i(x-t)\omega} dx \right) g(t) e^{-it\omega} dt$$

$$= \mathcal{F}[f(x)](\omega) \int_{-\infty}^{\infty} g(t) e^{-it\omega} dt$$

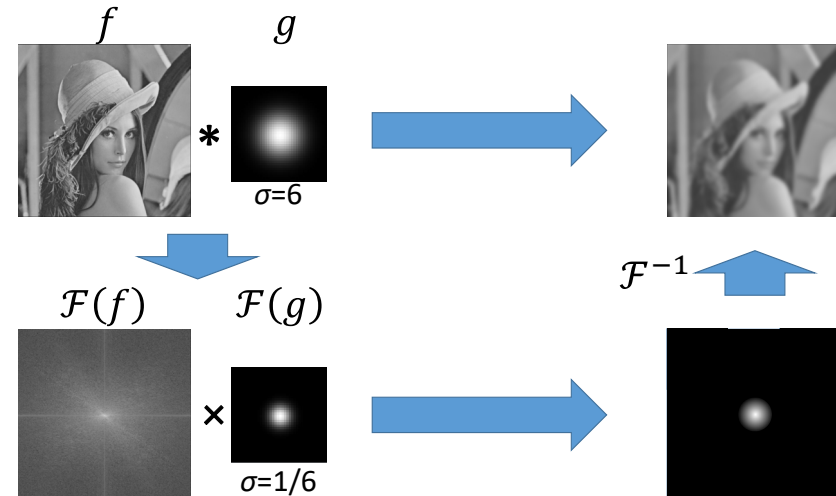
$$= \mathcal{F}[f(x)](\omega) \mathcal{F}[g(x)](\omega)$$

つまり,

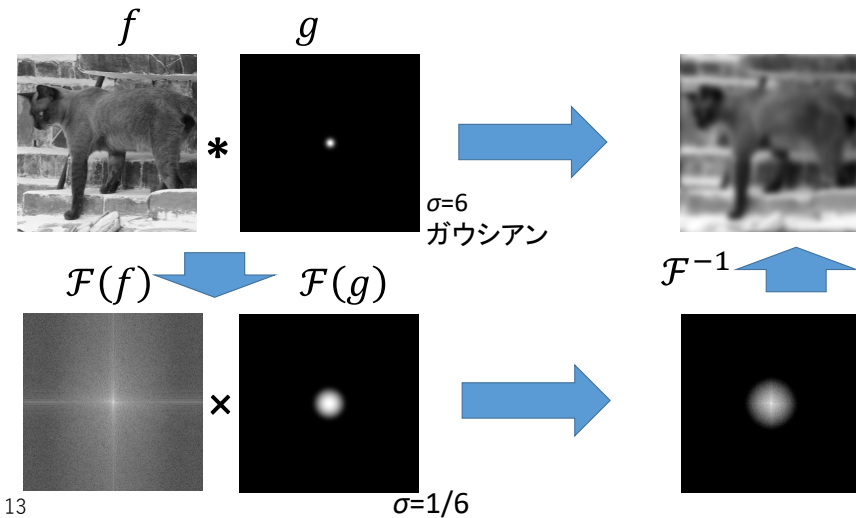
$$\mathcal{F}[f(x) * g(x)](\omega) = \mathcal{F}[f(x)](\omega) \mathcal{F}[g(x)](\omega)$$

畳み込み積分のフーリエ変換: $\mathcal{F}(f * g) = \mathcal{F}(f)\mathcal{F}(g)$

復習



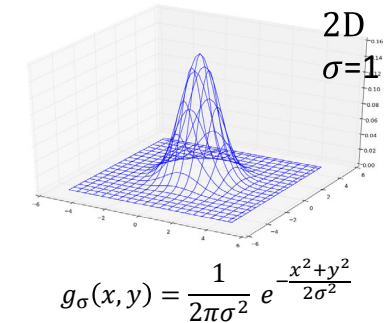
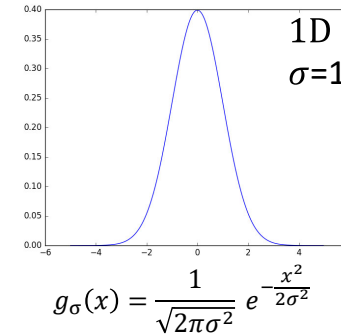
畳み込み積分のフーリエ変換: $\mathcal{F}(f * g) = \mathcal{F}(f)\mathcal{F}(g)$



復習

ガウシアンフィルタとは

ガウス関数により畳み込むフィルタのこと
低周波成分のみを通し画像を平滑化する効果がある（ローパスフィルタ）
画像処理において様々な場面で活躍する



14

ガウシアン of フーリエ変換はガウシアン

標準偏差 σ のガウス関数

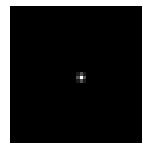
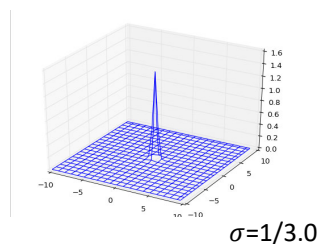
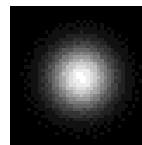
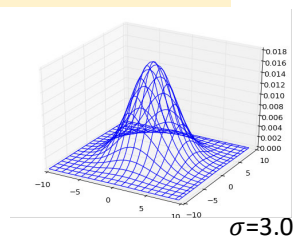
$$g_{\sigma}(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}$$

をフーリエ変換すると標準偏差が逆数のガウシアンになる

$$\mathcal{F}[g_{\sigma}(x)](\omega) = \int_{-\infty}^{\infty} g_{\sigma}(x) e^{-\omega x i} dx = e^{-\frac{\omega^2 \sigma^2}{2}}$$

または

$$\mathcal{F}[g_{\sigma}(x)](\omega) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} g_{\sigma}(x) e^{-\omega x i} dx = \frac{1}{\sqrt{2\pi}} e^{-\frac{\omega^2 \sigma^2}{2}}$$



15

復習

練習問題

(1) 右の画像中の赤い画素について、sobel filterを用いて『勾配(I_x, I_y)』『勾配強度の2乗($I_x^2 + I_y^2$)』『偏角 $\theta = \text{atan2}(I_y, I_x)$ 』を計算せよ。偏角については計算機等を利用してよい ※Pythonのmath.atan2(y, x)を利用するとよいです。

1	1	2	3	4
1	1	2	3	4
6	5	1	7	8
5	6	0	7	8
9	9	10	11	12

(2) Gaussian filterの標準偏差 σ を大きくすると、平滑化の強さはどうなるか。次のうち最も適切なものを選べ：

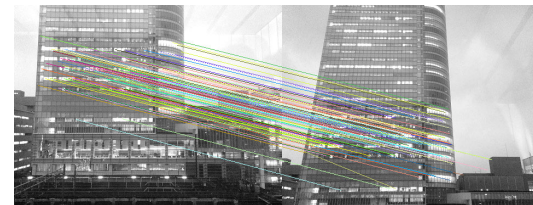
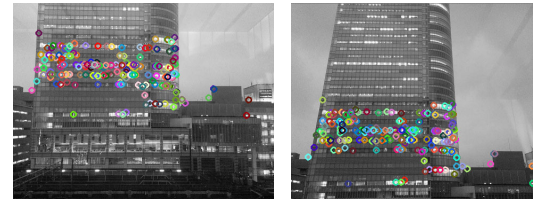
- A. 弱くなる（ぼかしが減る）
- B. 強くなる（ぼかしが増す）
- C. 変わらない

(3) 関数 $\frac{1}{\sqrt{2\pi 3^2}} e^{-\frac{x^2}{2 \times 3^2}}$ をフーリエ変換せよ

18

特徴点とは

特徴点とは



画像内から特徴的な場所を検出し似た特徴を持つ場所と対応付けしたい

→パノラマ合成, ステレオ視, 物体認識, VR (位置あわせ), etc

画像内から特徴的な点を検出する

検出した点の局所的な特徴を計算機が処理できる形で記述したい

- 局所特徴を多次元ベクトルで表現
- 拡大/回転に強い記述が理想 (拡大縮小・回転があっても特徴量が変化しない)

特徴ベクトルとは

SIFT特徴 Scale Invariant Feature Transform

- 画像2枚から特徴的な点を沢山抽出できたとしてどれとどれが似ているかを知りたい
- つまり, どれとどれが似た局所画像を持つか知りたい

→ 検出した特徴点の周囲の情報を, 比較できる形 (数値データ) に変換したい

!!!特徴ベクトル!!!

- 撮影条件によって対象は回転・拡大縮小・平行移動するので特徴点の位置・向き・サイズが変わる
 - 周囲が回転・拡大縮小しても似た特徴ベクトルを生成できる手法がほしい
- この条件を満たすSIFTが良く用いられてきた

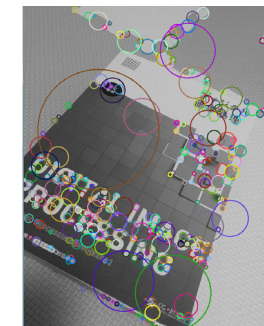
- 有名 & 頻繁に利用される特徴量のひとつ
- 周囲の特徴を128次元ベクトルで表現
- 回転・拡大縮小に堅固
 - 回転・拡大縮小があっても似た特徴ベクトルを出力できる

- 特徴ベクトルにすると局所領域の相違度を計算できる

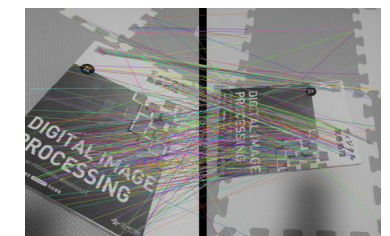
$$\text{相違度} = \sum_{i=1}^{128} (a_i - b_i)^2$$

※ a_i, b_i は特徴ベクトルの要素

※これは相違度の一例



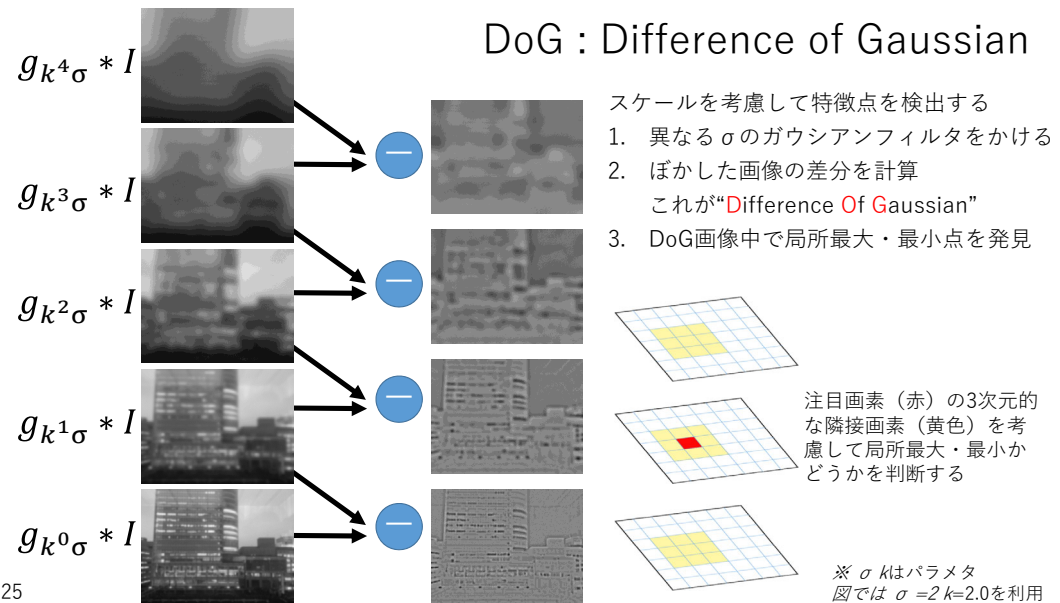
各点が128次元の特徴ベクトルを持つ



SIFT特徴

24

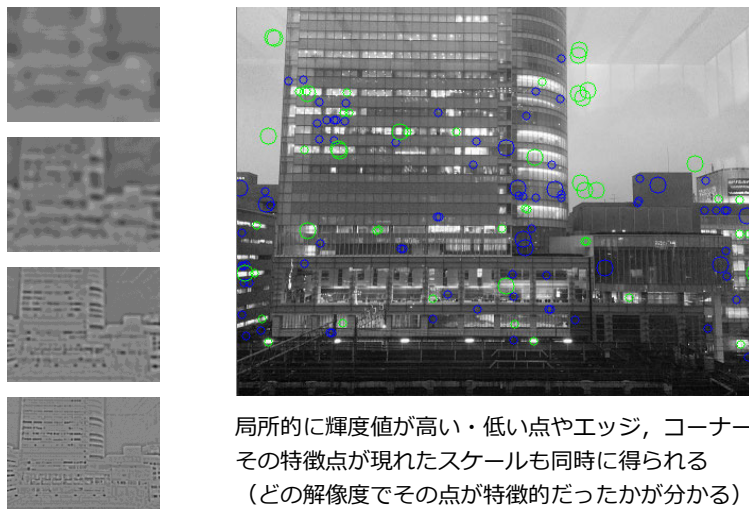
DoG : Difference of Gaussian



25

DoG : Difference of Gaussian

DoG.py



26

1. 特徴点検出

- DoGの極大・極小を特徴点とする
- Harris 行列を用いてエッジ点は除去
- 閾値処理でノイズ（極大値が小さい点）も除去

2. 方向検出

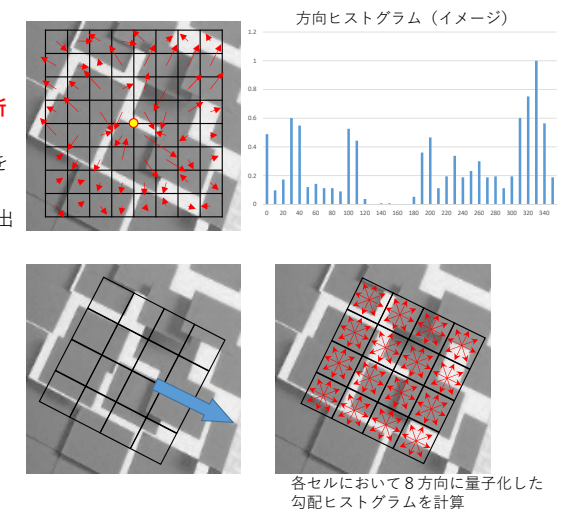
- 発見した特徴点において**そのサイズに合わせた局所領域**を考える
- 勾配ヒストグラムを生成（方向を36分割し、強度を中心からの距離で重み付け）
- ヒストグラムを正規化し強度が0.8以上の方向を検出（複数検出される→複数の特徴量を生成）

3. 特徴ベクトル計算

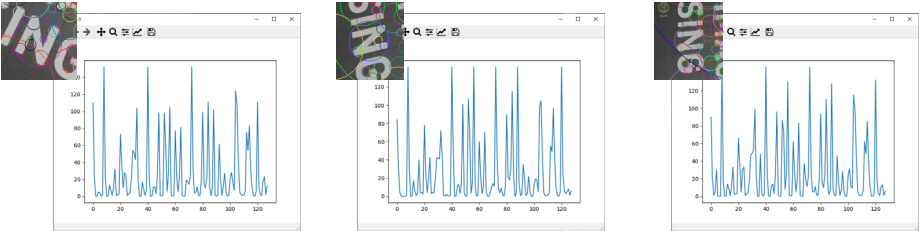
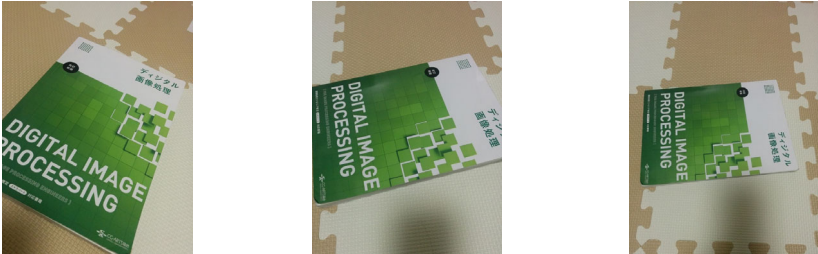
- 検出した方向に沿って局所領域を回転**
- 領域を4x4分割し、各領域内で勾配ヒストグラムを計算する
- 勾配ヒストグラムを特徴ベクトルとする
 - 勾配は8方向に量子化
 - 4*4*8 = 128次元ベクトルに
- 得られた特徴ベクトルを正規化（ベクトルの総和で割る）

27

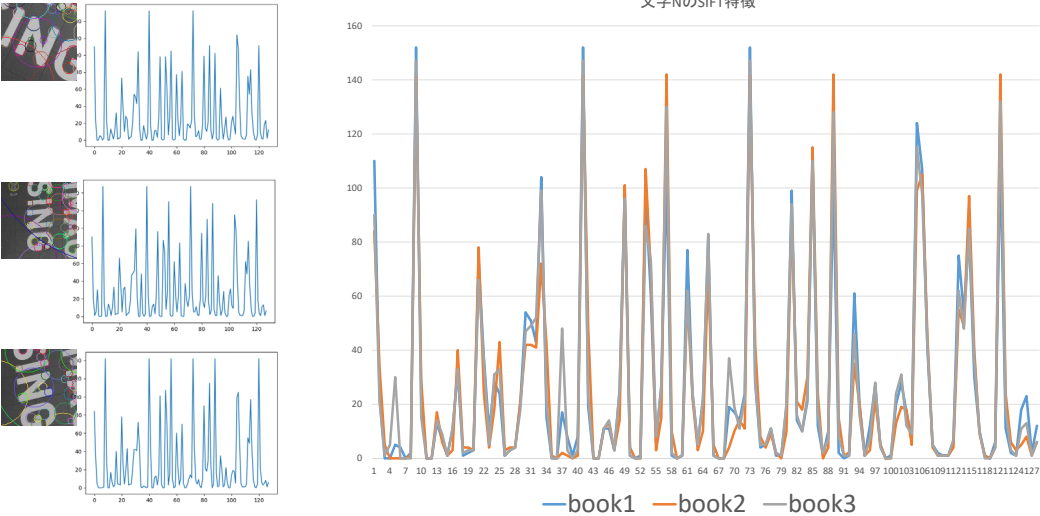
SIFT特徴



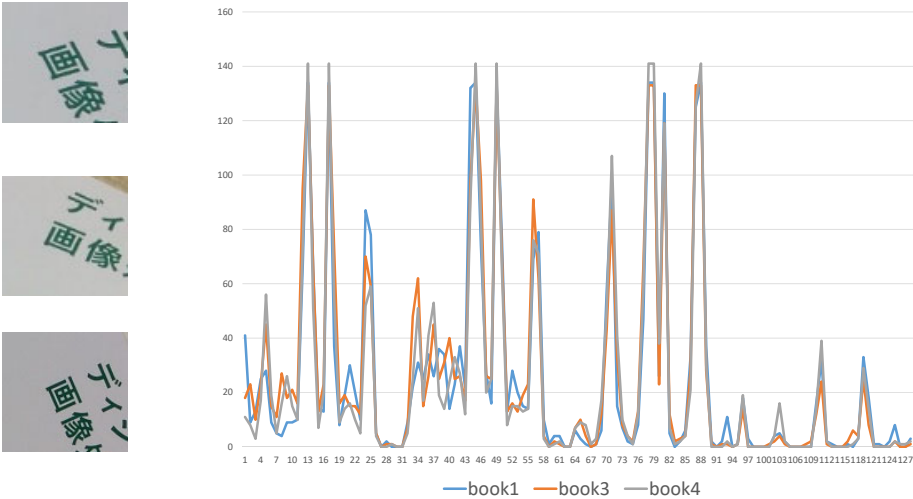
SIFT特徴点の例



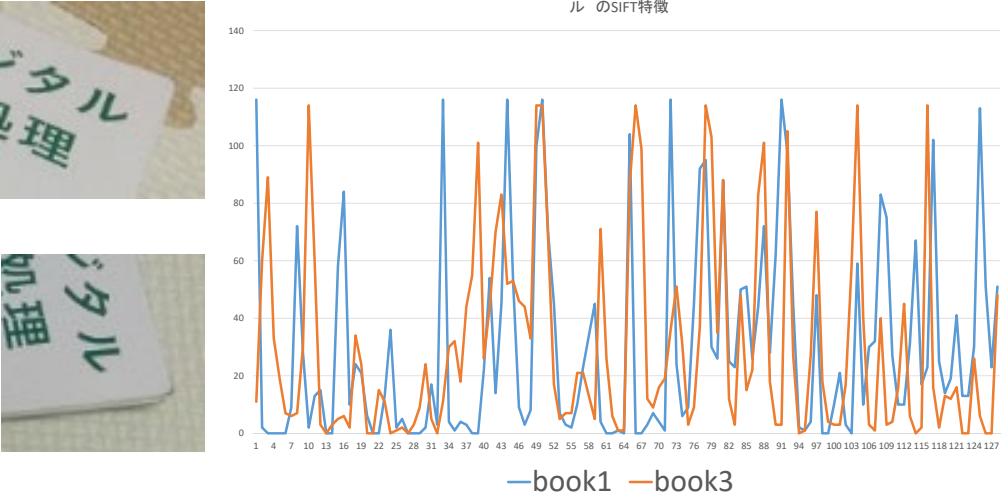
文字NのSIFT特徴



画 のSIFT特徴



ル のSIFT特徴



1. 特徴点検出

- DoGの極大・極小を特徴点とする
- Harris 行列を用いてエッジ点は除去
- 閾値処理でノイズ（極大値が小さい点）も除去

2. 方向検出

- 発見した特徴点においてそのサイズに合わせた局所領域を考える
- 勾配ヒストグラムを生成（方向を36分割し、強度を中心からの距離で重み付け）
- ヒストグラムを正規化し強度が0.8以上の方向を検出(複数検出される→複数の特徴量を生成)

3. 特徴ベクトル計算

- 検出した方向に沿って局所領域を回転
- 領域を4x4分割し、各領域内で勾配ヒストグラムを計算する
- 勾配ヒストグラムを特徴ベクトルとする
 - 勾配は8方向に量子化
 - $4 \times 4 \times 8 = 128$ 次元ベクトルに
- 得られた特徴ベクトルを正規化（ベクトルの総和で割る）

質問：
SIFT特徴は
なぜ拡大・回転について不変なのか？

※手順を覚えてほしいわけではなくて、
このように設計した特徴ベクトルが、なぜ拡大縮小と回転に対して不変（変化しにくい）となるかを説明できるようにしてほしい

SIFT特徴（実装）

```
# SIFT.py
img1 = cv2.imread("画像名.bmp", 0)
sift = cv2.xfeatures2d.SIFT_create()
key1, des1 = sift.detectAndCompute (img1, None )
```

Python & openCV環境だと上記の3行でSIFT特徴を検出できます

※key1 は特徴点の位置を保持する配列

※des1 は特徴点の特徴ベクトルを保持する配列

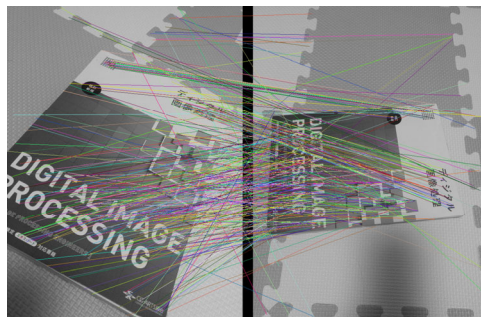
※『xfeatures2d.SIFT_create』を書き換えると色々な特徴量を試せます

最近はC++で全部書くのは流行らないみたい。

33 良い時代ですね。。。

まとめ: SIFT特徴

- 特徴ベクトルとは何かを解説した
 - 検出された特徴点同士を比較するため、特徴点周囲の局所領域をベクトルの形で表すもの。
 - 特徴ベクトルは、SIFT, BRIEF, ORB, SURF, AKAZEなど、沢山の種類がある
 - 特徴ベクトルは目的や対象画像の依存してよいものを選択すべき
- SIFT特徴
 - DoGの極値を特徴点として検出
 - 特徴点のスケールに応じた局所領域を考慮
 - 特徴点周囲の勾配方向に沿って局所窓を回転
 - 局所窓を4分割し、各領域の勾配ヒストグラムを特徴ベクトルとする



練習問題

ある画像の一部領域（3x3）の勾配をsobel filterで計算したところ右図のとおりとなった

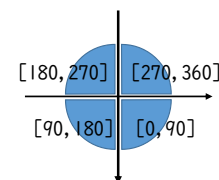
1) 中心画素の勾配強度の2乗を計算せよ

2) 勾配方向を4等分（[0, 90] [90, 180] [180, 270] [270, 360]）し、4つのビンを用意する。中心画素はどのビンに入るか求めよ。

3) 上記の4つのビンを利用し、この3x3画素の勾配ヒストグラムを計算せよ。ただし各画素の勾配強度の2乗を、対応すビンに投票すること

※ SIFTでは、勾配強度を投票しますが、ルートの計算が面倒なのでここでは2乗を投票しています。

$I_x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$	$\begin{pmatrix} 1 \\ 2 \end{pmatrix}$	$\begin{pmatrix} -2 \\ 1 \end{pmatrix}$
$\begin{pmatrix} 1 \\ 1 \end{pmatrix}$	$\begin{pmatrix} -3 \\ 1 \end{pmatrix}$	$\begin{pmatrix} 1 \\ -1 \end{pmatrix}$
$\begin{pmatrix} -2 \\ 2 \end{pmatrix}$	$\begin{pmatrix} -1 \\ 1 \end{pmatrix}$	$\begin{pmatrix} -1 \\ -1 \end{pmatrix}$



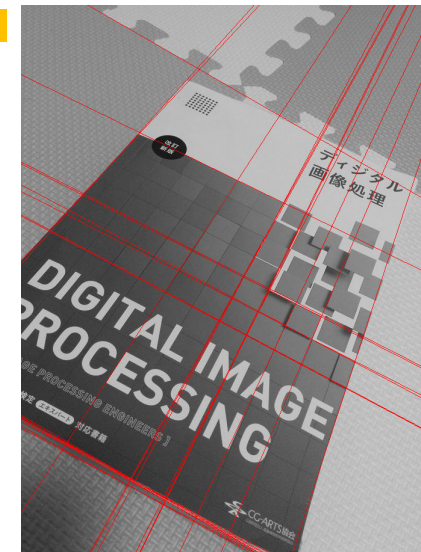
注) y軸は
下方向が正です

Hough変換

Hough変換とは

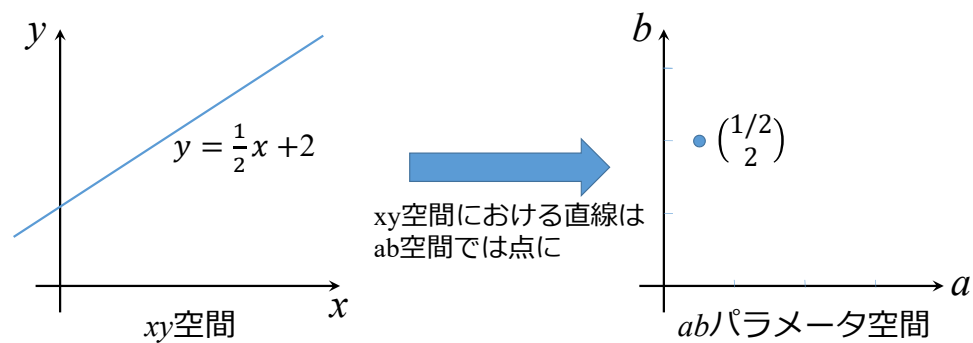
- 画像中から直線や円を検出する手法
- 直線や円の一部分が破損・劣化していても検出可能

Hough.py



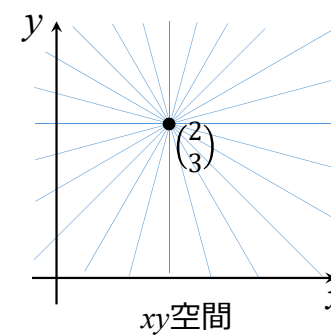
xy 空間と ab 空間

xy 空間における直線は『 $y = ax + b$ 』と表せる
直線の傾き a を横軸・ y 切片 b を縦軸にとる ab 空間を考える



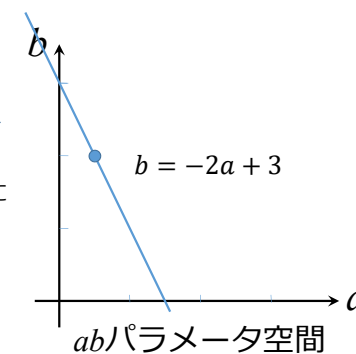
xy 空間と ab 空間

点(2,3)を通る直線群



xy空間における
(2,3)を通る直線群
ab空間では
直線: $b = -2a + 3$ に

$y = ax + b$ が(2,3) を通るので
 $b = -2a + 3$



Hough変換

入力：画像

出力：エッジ画素を通る直線群

※ エッジ画素：勾配強度が閾値以上の画素

1. 画像をエッジ画像変換（勾配強度を閾値処理）

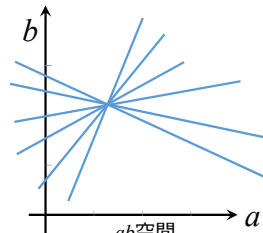
2. 各エッジ画素について…

- その画素を通る直線群は ab 空間で直線になる。その直線を ab 空間に描く

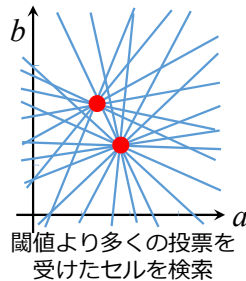
※ 実際は ab 空間を小さな画素に分割し、直線が通る画素値を1プラスする（投票）

3. 多くの直線が通る（投票が閾値以上の） ab 空間のセルを検索し、そのセルの現す直線を出力

- 直線は複数発見される



ab 空間
実装時はセルの配列で表現される



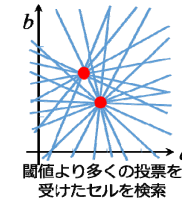
閾値より多くの投票を受けたセルを検索



41

• 先のアルゴリズムの問題点

- $y = ax + b$ として任意の直線を表現する場合、傾き a と切片 b のとりうる範囲は $[-\infty, \infty]$
- 多様な直線を検出するには、無限に広い ab 空間を用意して投票する必要がある…



閾値より多くの投票を受けたセルを検索



→ 解決法：直線を『 $\rho = x \cos \theta + y \sin \theta$ 』と表す

- 詳細は次ページ
- この表現を使うと θ と ρ の値の範囲を有限にできる

42

直線： $\rho = x \cos \theta + y \sin \theta$

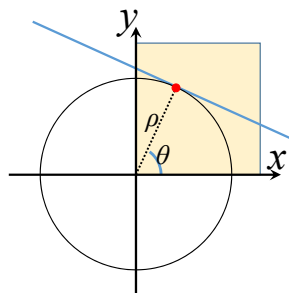
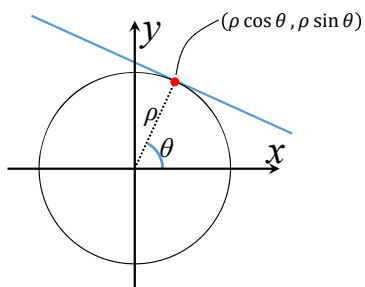
- この直線は、点 $(\rho \cos \theta, \rho \sin \theta)$ を通り、傾きは $\frac{-1}{\tan \theta}$

- 半径 ρ の円に接する

- θ が $[0, 2\pi]$ の範囲で動くとき全ての傾きをカバーできる（資料修正）

- ρ が $[0, A]$ の範囲で動くとき画像（下図の黄色領域）を通る全直線をカバーでき

※ $A = \sqrt{W^2 + H^2}$ は画像の対角の長さ

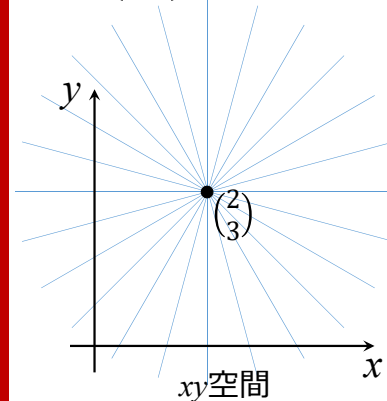


$W \times H$ の画像内に入る範囲で ρ と θ を動かす

43

直線を『 $\rho = x \cos \theta + y \sin \theta$ 』と表すと…

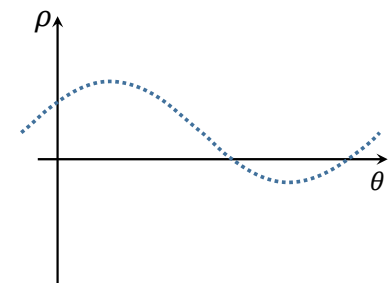
点(2,3)を通る直線群



$$\rho = 2 \cos \theta + 3 \sin \theta$$

$$\rho = \sqrt{13} \sin(\theta + \alpha)$$

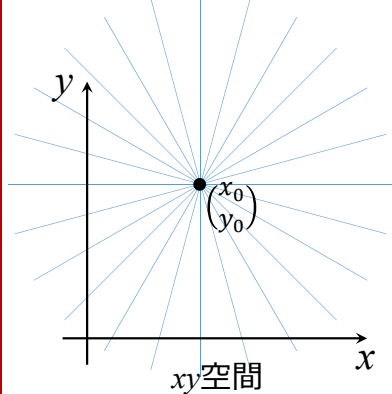
(2,3)を通る直線群は ρ 空間では $\rho = 2 \cos \theta + 3 \sin \theta$ という正弦波になる



44

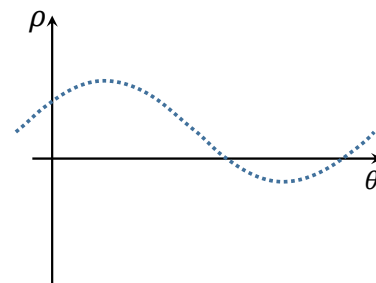
直線を『 $\rho = x \cos \theta + y \sin \theta$ 』と表すと…

点 (x_0, y_0) を通る直線群



$$\begin{aligned}\rho &= x_0 \cos \theta + y_0 \sin \theta \\ &= \sqrt{x_0^2 + y_0^2} \sin(\theta + \alpha)\end{aligned}$$

(x_0, y_0) を通る直線群は
 $\rho - \theta$ 空間では
 $\rho = x_0 \cos \theta + y_0 \sin \theta$
という正弦波になる



45

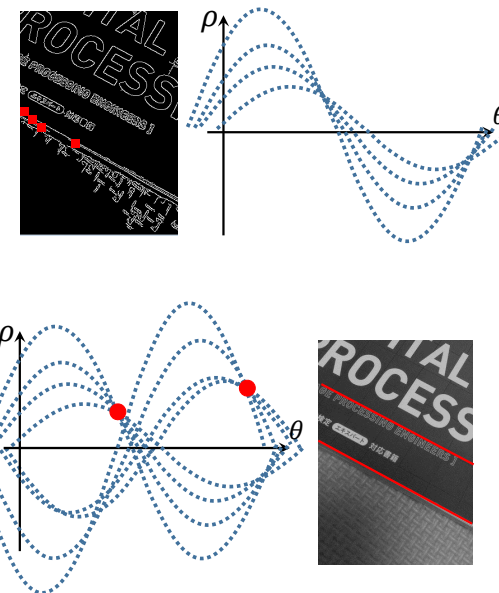
Hough変換

入力：画像

出力：エッジ画素を通る直線群

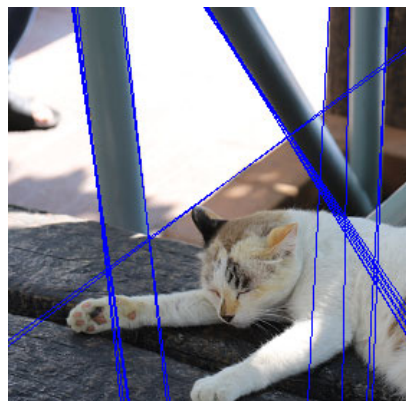
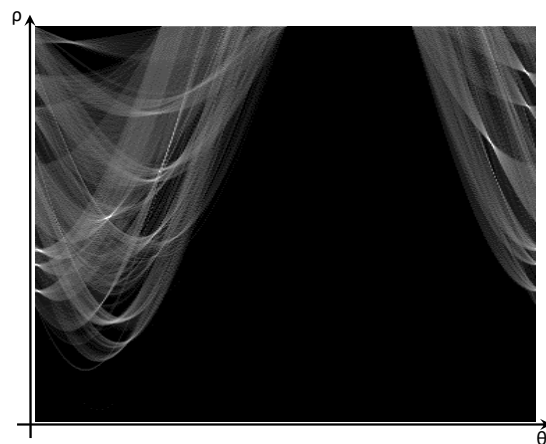
※ エッジ画素：勾配強度が閾値以上の画素

1. 画像をエッジ画像へ変換（勾配強度を閾値処理）
2. 各エッジ画素について…
 - エッジ画素を通る直線群は $\rho - \theta$ 空間で正弦波になるので、 $\rho - \theta$ 空間にその正弦波を描く
3. 多くの正弦波が通る（投票が閾値以上） $\rho - \theta$ 空間のセルをみつけ、対応する直線を出力
 - 直線は複数発見される



46

実際の $\rho - \theta$ 空間に投票した画像は以下の通り



47

Hough変換で円を検出する

直線とほぼ同じ方法で検出可能

- 円を $(x - c_x)^2 + (y - c_y)^2 = r^2$ と表現し
- 全画素 (x, y) を通るすべての円を c_x, c_y, r の三次元空間に投票
- c_x, c_y, r 空間にて極大値かつしきい値以上の (c_x, c_y, r) の組を発見する

参考

48

まとめ : Hough変換

- 画像中の直線や円を検出する手法

0. 直線（または円）を数式で表現する

1. 入力画像からエッジ画像を計算

2. 全てのエッジ画素について…

パラメータ空間の対応セルの値をプラス1する

（直線検出なら ρ θ 空間の正弦波を考える）

3. パラメータ空間において値の大きなセルを検索

そのセルが対応する直線を出力

